

Ohanyere Francis

+234 813 432 0174 — ohanyerefrancis395@gmail.com — github.com/ohanyere — linkedin.com/in/ohanyere-francis

Summary

Platform Engineer with production experience building scalable GitOps pipelines on AWS EKS, managing multi-cluster Kubernetes environments, and implementing observability (Prometheus/Grafana) and service mesh (Istio with Gateway API) for resilient cloud-native systems. Proven ability to lead platform enablement, drive incident response readiness, and embed FinOps guardrails into CI/CD workflows. Seeking to bring operational rigor and automation-first mindset to a mid-level Platform Engineering role.

Technical Skills

Cloud & Infrastructure: AWS (EKS, EC2, VPC, S3, IAM, Route53), Terraform, Kubernetes (Multi-cluster), Docker, Pulumi

Service Mesh & Networking: Istio (Gateway API), Cilium, Envoy, AWS Load Balancer Controller

CI/CD & GitOps: GitHub Actions, ArgoCD (ApplicationSets), Helm, Kustomize, Flux

Observability: Prometheus, Grafana, Alertmanager, Loki, Tempo, OpenTelemetry

Security & Compliance: Trivy, Checkov, OPA/Conftest, Kyverno, Pod Security Standards

FinOps & DR: Infracost, Velero, AWS Backup, Cost Allocation Tags, RPO/RTO Planning

Languages & Data: Go (Production), Python (Automation), Bash, PostgreSQL, Redis

Experience

ICT Slides

2024 – Present

Platform Engineer | Early-stage EdTech platform

Anambra State, Nigeria

- Led platform enablement for 15+ developers: taught Kubernetes, GitOps, and observability best practices; managed multi-tenant EKS clusters used in production training environments.
- Designed and maintained monitoring stack (Prometheus/Grafana/Alertmanager) with alerting rules for cluster health, node capacity, and application SLOs, reducing mean-time-to-detection (MTTD) for issues by 40%.
- Implemented Istio service mesh with Gateway API for traffic management, mTLS encryption, rate limiting, and canary deployments across staging/production namespaces.
- Managed multi-cluster deployments (dev/stage/prod) using ArgoCD with ApplicationSet patterns, ensuring environment parity and safe progressive rollouts.
- Owned incident response playbook: defined SLOs/SLIs, created runbooks for common failures (pod crashes, node failures, network issues), and led post-mortems to drive reliability improvements.
- Built backend services using Go and Node.js with PostgreSQL for persistence and Redis for caching, supporting internal platform tooling workflows.
- Provisioned reproducible infrastructure using Terraform, reducing configuration drift across environments by 70% and enabling self-service platform capabilities.

Key Projects

Cluster Meter | Go, Kubernetes, Prometheus, Grafana, EKS, ArgoCD

Github Link

- Addressed lack of visibility into Kubernetes resource usage and inefficient workload allocation across multi-cluster environments.
- Built a Go-based CLI and API service to collect cluster metrics via Kubernetes API and analyze resource pressure with real-time dashboards in Grafana.
- Extended to support multi-cluster metric aggregation via Kubernetes Federation API and Prometheus federated scraping.
- Implemented runtime checks for CPU, memory, and pod saturation with Alertmanager notifications, reducing cluster outages in test environments by 60%.
- Enabled identification of inefficient workloads and right-sizing recommendations, improving overall cluster utilization by 35%.

EKS Disaster Recovery Platform | Kubernetes, Velero, AWS S3, Terraform

Github Link

- Addressed risk of workload loss and absence of validated disaster recovery strategy in Kubernetes environments.
- Implemented automated backup and restore workflow using Velero with S3-backed storage for persistent cluster state and etcd snapshots.
- Configured scheduled backup policies with retention rules aligned with RPO (4 hours) and RTO (≤30 minutes) objectives.
- Documented and socialized DR runbook used during simulated incident response drills and failure scenarios.
- Achieved complete workload recovery with zero data loss and validated disaster recovery readiness across 3 simulated failure scenarios.

FinOps Cost Guardrail Pipeline | Terraform, Infracost, GitHub Actions, OPA

Github Link

- Addressed lack of cost visibility and risk of expensive infrastructure changes before deployment to production.
- Built CI-based guardrail using Terraform plan and Infracost to estimate cost impact in pull requests with Slack/Teams notifications.
- Integrated cost checks into CI pipeline with enforced thresholds (\$100/month increase limit) and multi-environment cost comparison.
- Enforced policy-as-code with OPA/Conftest alongside cost checks for compliance-ready deployments.
- Prevented high-cost infrastructure changes during pull requests, avoiding \$2,000/month in potential over-provisioning.

Key Achievements

- Reduced manual deployment effort by 70% through GitOps automation with ArgoCD and GitHub Actions.
- Achieved zero data loss in disaster recovery validation across 3 simulated failure scenarios with ≤30 minute recovery time.
- Enabled cost-aware infrastructure changes, preventing \$2,000/month in potential over-provisioning through CI-enforced guardrails.
- Mentored 15+ developers on cloud-native practices (Kubernetes, GitOps, observability), improving team velocity and platform adoption.
- Implemented service mesh (Istio) with Gateway API, enabling zero-downtime deployments and mTLS security across 5+ microservices.

Education

Chukwuemeka Odumegwu Ojukwu University — B.Sc. Computer Science	2020 – 2024
ALX Africa — Software Engineering Fellowship	2023 – 2024

Certifications

- Certified Kubernetes Administrator (CKA) — *In Progress*
- AWS Certified Solutions Architect — Associate — *In Progress*
- Istio Service Mesh Fundamentals — *Self-Study*